

SafePay V1 Decision Register

Document type: Authoritative approved requirement and contract register

Project: SafePay V1 Prototype

Status: Phase 1.1 approved

Approval date: 2026-09-09

Applies to: Database, backend, REST APIs, Oracle JET PWA, testing, and local deployment

1. Purpose

This document consolidates the decisions approved during Phase 1.1A through Phase 1.1G. It is the persistent project reference for the SafePay V1 implementation.

It exists to prevent contradictions, requirement drift, accidental restoration of superseded PRD requirements, and implementation based on assumptions. Before starting or continuing any database, backend, API, frontend, test, or deployment phase, **the approved requirements in this document must be consulted.**

The baseline PRD and implementation plan remain important supporting resources. Where they conflict with a later approved decision recorded here, this register controls unless the user subsequently gives a newer instruction.

2. Authority and Change-Control Contract — Phase 1.1A

2.1 Requirement precedence

When project resources disagree, apply the following order:

1. The user's latest explicit instruction or approval.
2. Earlier user-approved decisions that have not subsequently been replaced.
3. The root `AGENTS.md` instructions.
4. This approved SafePay V1 Decision Register.
5. `Updated_Implementation_Full_Plan.md`.
6. `SafePay_PRD.md` and `SafePay_PRD.pdf`.
7. Earlier diagrams, research, brainstorming, and supporting material.

A newer decision replaces only the affected requirement. It does not silently invalidate unrelated approved requirements.

2.2 Change-control procedure

For every proposed on-the-go change:

1. Identify the exact requirement being changed.
2. Identify affected database, backend, API, frontend, security, testing, and deployment areas.
3. Explain whether the change is compatible with existing approved behaviour.
4. Explain whether it causes migration, rework, or new dependencies.
5. Classify it as an immediate V1 change or deliberately deferred work.
6. Obtain explicit user approval before treating it as final.
7. Record the approved change in this register.

2.3 Guided-development rule

Implementation proceeds as a teacher-and-learner exercise:

1. Explain the concept and why SafePay needs it.
2. Identify the exact manually created folder and file.
3. Provide the precise content for the user to paste.
4. Explain significant annotations, relationships, security controls, and logic.
5. Compile or test the smallest completed unit.
6. Diagnose and resolve current errors.
7. Record genuinely future-dependent work in a carry-forward register.
8. Obtain approval before progressing to the next meaningful phase.

Except where the user gives a specific one-time authorization, the agent must not directly create or modify implementation files. Implementation changes are performed manually by the user using guided instructions.

2.4 Product scope boundary

SafePay V1 is:

A simulated, risk-adaptive pre-settlement transaction-control layer operating on its own simulated accounts and internal ledger.

SafePay may:

- Delay a simulated payment before settlement.
- Allow cancellation before simulated release or settlement.
- Hold a simulated high-value payment for verification and Risk Officer review.
- Reserve simulated funds while a payment is pending.
- Record simulated settlement using balanced double-entry ledger entries.

SafePay **must** not:

- Claim to intercept a live UPI, IMPS, NEFT, or RTGS transaction already in flight.
- Claim to reverse a completed external interbank settlement.
- Move real money.
- Represent an internal clearing credit as proof that an external beneficiary was actually paid.
- Present itself as a production KYC, AML, or fraud-detection system.

All relevant UI screens must clearly use simulated-safe language such as “Simulated Settlement” and display an appropriate simulation indicator.

2.5 Confirmed V1 exclusions

The following are excluded from V1 risk evaluation and must not have dormant V1 database columns, request fields, services, or screens:

- Device fingerprint or device trust.
- Location or geolocation.
- Beneficiary age as a risk signal.
- Transaction velocity or frequency.
- Historical amount anomaly.
- Rolling-average comparison.
- Weighted multi-signal scoring.
- AI-based classification or recommendation.

These may return only through separately approved V2 feature work covering schema, APIs, consent, retention, security, UI, and testing.

3. Amount-Risk and Protection Contract — Phase 1.1B

3.1 Deterministic amount-only classification

SafePay V1 uses the payment value only. The same valid amount evaluated under the same policy version must always receive the same tier and protection action.

Every valid amount must match exactly one band.

Payment amount	Risk tier	Protection duration	Action
₹1.00 to ₹5,000.00 inclusive	LOW	0 seconds	Immediate simulated release and settlement
Above ₹5,000.00 and up to ₹25,000.00 inclusive	MEDIUM	10 seconds	Protection window with customer Undo
Above ₹25,000.00 and up to ₹1,00,000.00 inclusive	HIGH	60 seconds	Protection window, stronger warning, and customer Undo
Above ₹1,00,000.00	VERY_HIGH	No timer	Hard hold, OTP verification, and Risk Officer review

3.2 Boundary cases

Input	Expected result
Null or missing	Validation failure; no transaction created
Negative amount	Validation failure; no transaction created
₹0.00	Validation failure; no transaction created
₹0.01 to ₹0.99	Validation failure; below minimum
₹1.00	LOW
₹4,999.99	LOW
₹5,000.00	LOW
₹5,000.01	MEDIUM
₹24,999.99	MEDIUM
₹25,000.00	MEDIUM
₹25,000.01	HIGH
₹99,999.99	HIGH
₹1,00,000.00	HIGH
₹1,00,000.01	VERY_HIGH

The boundary values above are mandatory risk-engine and API tests.

3.3 Money-validation rules

- V1 currency is INR only.
- Minimum transaction amount is ₹1.00.
- Amount is mandatory and must be positive.
- Amount may contain no more than two decimal places.

- Values such as 5000.001 must not be rounded silently.
- Java uses `BigDecimal`, never `double` or `float`.
- Oracle stores monetary values as `NUMBER(18,2)`.
- Commas and the rupee symbol are display formatting only.
- Risk comparisons use exact decimal comparison.
- The risk engine does not hard-code an overall payment maximum. Any future maximum is configurable; account availability and schema precision still apply.

3.4 Excess decimal places

The `Oracle JET frontend must stop the customer before submission` and show a short message such as:

Enter an amount with no more than two decimal places, for example ₹5,000.01.

When frontend validation catches this:

- No API request is sent.
- No transaction record is created.
- No transaction audit record is created.
- No transaction-exception record is created.

The backend independently enforces the same rule in case a caller bypasses the frontend. It rejects the request before transaction persistence and does not classify it as a rejected or cancelled transaction.

3.5 Risk result contract

The amount-risk engine returns:

- Risk tier.
- Nullable risk score.
- Protection duration.
- Matched amount-band identity.
- Policy/model version.
- Customer-safe explanation.
- Evaluation timestamp.

For V1, `riskScore` remains null. SafePay must not invent a numeric score when no weighted scoring model ran.

An example MEDIUM result contains the tier, a null score, ten-second duration, policy version `AMOUNT_ONLY_V1`, and an explanation that the amount matched the configured MEDIUM band.

3.6 Policy versioning

- The V1 amount policy has a stable identity such as `AMOUNT_ONLY_V1`.
- Every assessed transaction stores the policy version used for that assessment.

- A later policy applies only to future assessments.
- Existing protected, held, cancelled, failed, or settled transactions are never silently reclassified.
- Old policy versions remain queryable for audit history.
- Activation of a new policy requires an explicit approved action.

3.7 Timer rules

For MEDIUM and HIGH transactions:

```
protectedUntil = authoritative server/database start time + configured duration
```

- The backend creates and enforces the deadline.
- The browser only displays remaining time.
- Closing or refreshing the browser does not pause or extend protection.
- Network disconnection does not extend the deadline.
- The server decides whether an Undo request arrived in time.
- The timer begins only after authorization, risk assessment, fund reservation, and successful persistence of the protected state.
- MEDIUM and HIGH may release automatically after expiry if still eligible.
- VERY HIGH never releases because time passed.

3.8 Tier-specific behaviour

LOW: no timer, unnecessary warning, or OTP; proceed immediately to simulated settlement.

MEDIUM: show a 10-second protection window, permit customer cancellation, and automatically continue after authoritative expiry.

HIGH: show a 60-second protection window with stronger warning, permit customer cancellation, and automatically continue after authoritative expiry.

VERY_HIGH: place on an untimed hard hold, require OTP, route successful verification to Risk Officer review, and settle only following an eligible approval.

3.9 Threshold interpretation

The approved thresholds are SafePay V1 product rules. They must not be presented as a universal limit shared by every Indian bank, UPI category, or payment rail.

Payments above ₹1,00,000 remain valid SafePay simulation requests and enter VERY_HIGH handling. They are not automatically rejected. V1 does not separately implement rail-specific limits.

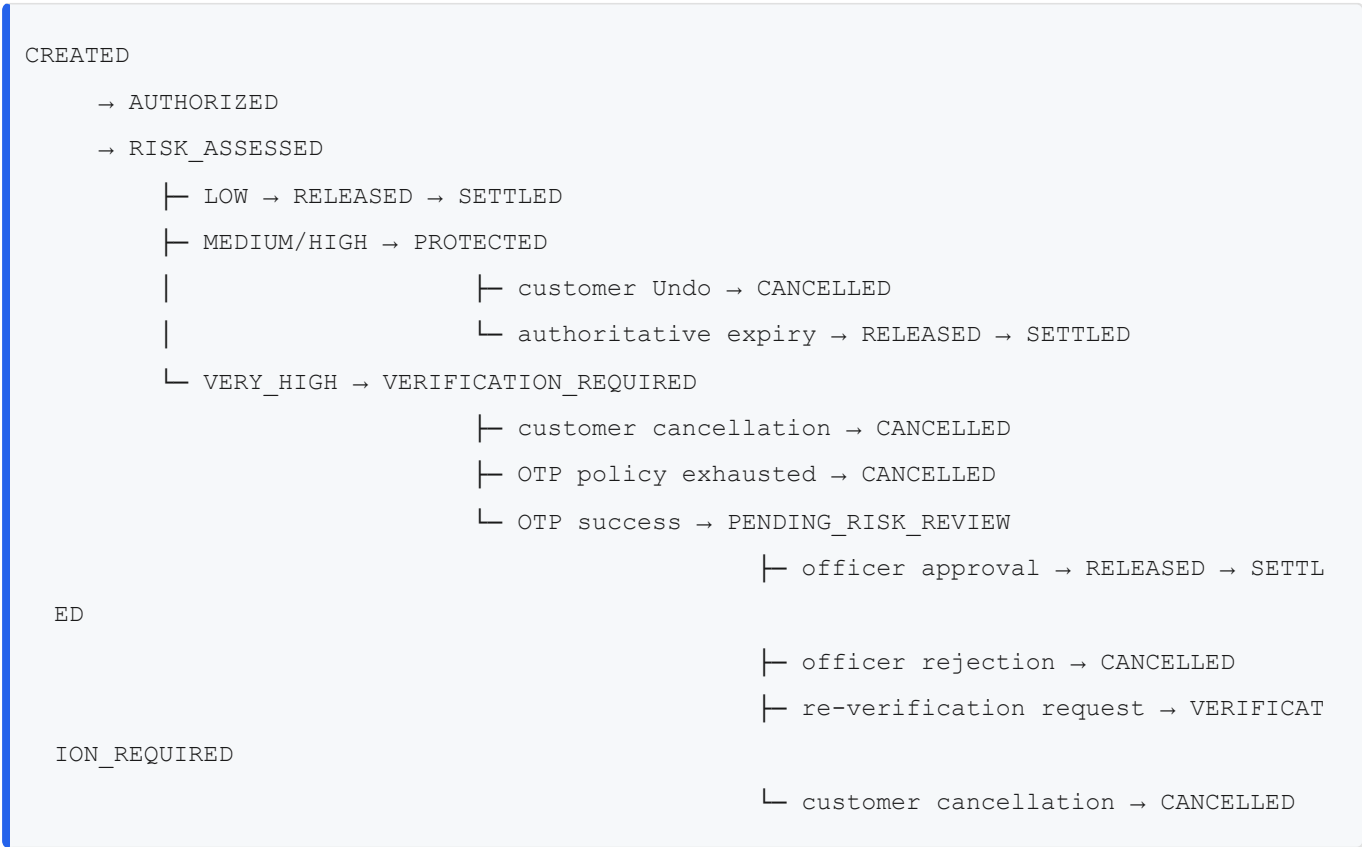
4. Transaction State-Transition Contract — Phase 1.1C

4.1 Approved transaction states

State	Meaning
CREATED	Payment instruction exists, but customer authorization is incomplete
AUTHORIZED	Customer explicitly confirmed payment intent
RISK_ASSESSED	V1 amount-risk result was evaluated and recorded
PROTECTED	MEDIUM or HIGH payment is inside its Undo window
VERIFICATION_REQUIRED	VERY_HIGH payment is waiting for OTP verification
PENDINGRISKREVIEW	OTP succeeded and the payment awaits a Risk Officer decision
RELEASED	All protection requirements passed and settlement is permitted
SETTLED	Simulated ledger posting completed successfully
CANCELLED	Payment was stopped before settlement
FAILED	Payment cannot complete because of a definitive business or processing failure

SETTLED, CANCELLED, and FAILED are terminal. They cannot return to an active state.

4.2 Main state flow



4.3 Transition matrix

Current state	Trigger	Required condition	Next state
CREATED	<u>Customer authorizes</u>	<u>Valid owner, account, beneficiary, and amount</u>	AUTHORIZED
CREATED	Customer cancels	Transaction belongs to customer	CANCELLED
AUTHORIZED	Risk evaluation completes	Valid active V1 policy	RISK_ASSESSED
AUTHORIZED	Definitive processing failure	Failure safely recorded	FAILED
RISK_ASSESSED	Tier is LOW	<u>Assessment and reservation operation succeed</u>	RELEASED
RISK_ASSESSED	Tier is MEDIUM/HIGH	<u>Reservation and deadline are saved</u>	PROTECTED
RISK_ASSESSED	Tier is VERY_HIGH	<u>Reservation and hard-hold requirement are saved</u>	VERIFICATION_REQUIRED
PROTECTED	Customer cancels	Server confirms deadline has not expired	CANCELLED
PROTECTED	Deadline expires	Transaction is still PROTECTED	RELEASED
VERIFICATION_REQUIRED	OTP succeeds	Correct, valid, unused OTP	PENDINGRISKREVIEW
VERIFICATION_REQUIRED	Customer cancels	Still awaiting verification	CANCELLED
VERIFICATION_REQUIRED	OTP policy exhausted	Attempt/issue policy exhausted	CANCELLED
PENDINGRISKREVIEW	Officer approves	Authorized officer and pending review	RELEASED
PENDINGRISKREVIEW	Officer rejects	Authorized officer and reason supplied	CANCELLED
PENDINGRISKREVIEW	Officer requests verification	Authorized officer and reason supplied	VERIFICATION_REQUIRED
PENDINGRISKREVIEW	Customer cancels	Approval has not already won	CANCELLED

Current state	Trigger	Required condition	Next state
RELEASED	Settlement succeeds	<u>All financial operations succeed atomically</u>	SETTLED
RELEASED	Definitive settlement failure	No partial settlement survives	FAILED

4.4 Review status versus transaction state

APPROVED and REJECTED are not payment states. Risk review uses separate statuses:

- PENDING.
- APPROVED.
- ~~REJECTED.~~
- REVERIFICATION_REQUESTED.

An approved review permits `PENDING_RISK_REVIEW → RELEASED`. A rejected review produces `PENDING_RISK_REVIEW → CANCELLED`. A re-verification request produces `PENDING_RISK_REVIEW → VERIFICATION_REQUIRED`.

4.5 Cancellation rules

A customer may normally cancel in:

- CREATED.
- PROTECTED.
- VERIFICATION_REQUIRED.
- `PENDINGRISKREVIEW`.

Customer cancellation is not exposed as a normal UI action during the short internal `AUTHORIZED` and `RISK_ASSESSED` stages. Cancellation is prohibited after RELEASED or SETTLED and after another concurrent action already won.

4.6 Concurrency rules

When customer Undo races timer expiry, exactly one transition may win:

- `PROTECTED → CANCELLED`; or
- `PROTECTED → RELEASED`.

When customer cancellation races Risk Officer approval, exactly one may win:

- `PENDINGRISKREVIEW → CANCELLED`; or
- `PENDINGRISKREVIEW → RELEASED`.

The losing request receives a state-conflict response containing or enabling retrieval of the authoritative state. JPA versioning, conditional updates, fresh deadline checks, transaction boundaries, and idempotency protect these operations. UI button disabling is not a financial control.

4.7 OTP re-verification

- An earlier OTP becomes invalid when re-verification is requested.
- A new challenge is required.
- The payment returns to VERIFICATION_REQUIRED.
- The review preserves who requested re-verification and why.
- Successful re-verification returns the payment to review.
- Successful re-verification does not settle or approve the payment automatically.

4.8 Temporary and definitive failure

Temporary failures, such as a brief database interruption, scheduler interruption, or restart, retain the last committed safe state. Any reservation remains intact and an idempotent retry may occur.

Definitive failures, such as insufficient available balance, a permanently inactive source account, an unsatisfied financial invariant, or exhausted retry policy, may move the transaction to FAILED. Partial financial changes must not survive.

4.9 State invariants

- A transaction has exactly one current state.
- Every transition records actor and timestamp.
- Risk assessment occurs before protection or release.
- MEDIUM and HIGH cannot bypass PROTECTED.
- VERY_HIGH cannot bypass OTP and Risk Officer review.
- Only RELEASED may settle.
- A transaction settles no more than once.
- SETTLED requires a complete balanced ledger posting.
- CANCELLED produces no settlement ledger entries.
- Terminal states never reopen.
- Clients never submit a desired transaction state.
- Every successful state change produces an audit event.
- Repeated requests never duplicate effects.

4.10 Lifecycle timestamps

The transaction model supports relevant nullable lifecycle timestamps:

- createdAt.
- authorizedAt.
- riskAssessedAt.
- protectedUntil.
- verificationCompletedAt.
- releasedAt.

- settledAt.
- cancelledAt.
- failedAt.
- updatedAt.

LOW has no protected deadline. VERY_HIGH uses verification/review timing rather than an auto-release deadline. CANCELLED has no settlement timestamp.

4.11 Transition auditing

Critical transition records identify:

- Actor.
- Previous state.
- Requested action.
- Resulting state when successful.
- Safe rejection reason when unsuccessful.
- Timestamp.
- Correlation/request identifier.

Frontend-only input validation does not create transaction audit events.

5. Settlement and Double-Entry Ledger Contract — Phase 1.1D

5.1 Balance concepts

SafePay distinguishes:

- **Current balance:** funds recorded in the simulated account.
- **Reserved amount:** funds committed to unsettled payments.
- **Available balance:** current balance minus reserved amount.

The invariant is: `available balance = current balance - reserved amount`

Current balance, reserved amount, and available balance must never become negative through an accepted operation.

5.2 Reservation lifecycle

Event	Financial effect
Transaction creation	No reservation
Authorization starts	No reservation yet
Validation/risk evaluation fails	No reservation
Risk assessment succeeds	Verify available balance and reserve payment amount
LOW release	Reservation is consumed during immediate settlement
MEDIUM/HIGH becomes PROTECTED	Reservation remains active
VERY_HIGH requires verification	Reservation remains active
Re-verification is requested	Reservation remains active
Officer approves	Reservation remains until settlement
Customer cancels	Reservation is released
Officer rejects	Reservation is released
Definitive failure	Reservation is released
Temporary failure	Reservation and last safe state remain
Settlement succeeds	Reservation is consumed and current balance is debited

Reservation and corresponding state changes occur within the same database transaction.

5.3 Reservation-bearing states

The amount remains reserved during:

- PROTECTED.
- VERIFICATION_REQUIRED.
- PENDINGRISKREVIEW.
- RELEASED.

CREATED, CANCELLED, FAILED, and SETTLED do not retain a reservation. AUTHORIZED and RISK_ASSESSED are brief processing states; the reservation is created atomically when the transaction enters its appropriate next path.

5.4 Insufficient available balance

SafePay verifies:

```
current balance - reserved amount >= payment amount
```

If false:

- No reservation is created.

- No ledger entry is created.
- The transaction does not enter protection or review.
- The transaction becomes FAILED.
- A safe reason such as `INSUFFICIENT_AVAILABLE_BALANCE` is recorded.

The check must be concurrency-safe. Two competing payments cannot reserve more than the available amount. A conditional update, account versioning, appropriate locking, and a fresh balance check enforce one-winner behaviour.

5.5 Double-entry settlement

Every successful settlement posts equal debit and credit values.

For an external beneficiary simulation:

Line	Account	Entry type	Amount
1	Customer source account	DEBIT	Payment amount
2	SafePay outbound-clearing account	CREDIT	Payment amount

Total debits must equal total credits.

The external beneficiary is not directly credited because V1 has no real payment-rail integration. The internal clearing credit represents simulated release for external submission, not proof of receipt.

5.6 Atomic settlement unit

Settlement conceptually performs:

1. Re-read and lock the payment.
2. Confirm it is still RELEASED.
3. Confirm the reservation exists logically.
4. Lock source and clearing accounts in a consistent order.
5. Revalidate account status and reserved amount.
6. Reduce customer current balance.
7. Reduce customer reserved amount.
8. Increase clearing-account balance.
9. Insert customer debit ledger line.
10. Insert clearing credit ledger line.
11. Validate equal totals.
12. Change payment to SETTLED.
13. Append settlement audit event.
14. Create notification/outbox information where required.
15. Commit as one transaction.

If any required operation fails, the whole unit rolls back. No partial debit, credit, balance update, or SETTLED state may survive.

5.7 Ledger posting identity

Every settlement has one posting group. Uniqueness uses:

```
(source_system, posting_group_key, posting_line_no)
```

Line numbers allow the legitimate debit and credit to share the same posting identity while preventing duplicate copies. The request idempotency key must not be uniquely constrained by itself on LEDGER_ENTRY because both balanced lines belong to the same request.

5.8 Ledger immutability

Posted ledger entries cannot be edited or deleted. Amount, account, direction, and posting identity are immutable.

Protection includes:

- No application update/delete methods for ledger history.
- Restricted database grants.
- Database protection against updates/deletes.
- Audit and reconciliation tests.

Future corrections use new compensating entries referencing the original posting. They never rewrite historical entries.

5.9 Ledger status and exception separation

Successful lines are inserted as POSTED. Failed attempts do not create half-complete or editable ledger pairs. Failure information belongs in TRANSACTION_EXCEPTION rather than in incomplete ledger data.

5.10 Balance reconstruction

The account table holds the latest operational balance. The immutable ledger is the history used to reconstruct and verify it:

```
expected balance = opening balance + posted credits - posted debits
```

Stored and reconstructed values must agree.

5.11 Opening balances

Seeded balances require a balanced origin. Opening balances are posted against a system control account such as SAFEPAYOPENINGBALANCE_CONTROL rather than appearing without ledger history.

5.12 Cancellation and failure effects

Cancellation before settlement:

- Creates no payment-settlement ledger pair.
- Releases any reservation.
- Produces CANCELLED.
- Records actor and reason.
- Appends audit and notification events.

Temporary settlement failure rolls back the attempt, retains RELEASED and its reservation, records the failed attempt safely, and permits idempotent retry.

Definitive settlement failure rolls back partial work, releases the reservation, produces FAILED, records the reason, and creates no completed settlement pair.

5.13 VERY_HIGH reservation

A VERY_HIGH payment remains reserved during verification and review. Customer cancellation or officer rejection releases it. Re-verification preserves it. Approval permits settlement but does not itself debit the account. There is no automatic timer-based release or automatic expiry in V1.

5.14 Reconciliation invariants

- Every SETTLED payment has exactly one settlement posting group.
 - Each posting group contains the required debit and credit lines.
 - Debit and credit totals match.
 - CANCELLED payments have no settlement posting.
 - FAILED payments have no completed settlement pair.
 - A payment has no duplicate settlement group.
 - Stored account balances match ledger-derived balances.
 - Reserved totals match eligible unsettled payments.
 - Clearing credits match settled external-beneficiary payments.
-

6. Users, Roles, and Permissions Contract — Phase 1.1E

6.1 Authentication and authorization

Authentication establishes who is making a request. Authorization determines whether that authenticated actor may perform a particular action on a particular resource.

A valid login never grants unrestricted access. SafePay must enforce both role permissions and resource ownership.

6.2 Approved application roles

Role	Purpose
CUSTOMER	Owns simulated accounts, beneficiaries, and payments
RISK_OFFICER	Reviews eligible VERY_HIGH payments
SYSTEM_ADMIN	Manages system access and operational administration
AUDITOR	Performs read-only audit and reconciliation review

Corporate MAKER and CHECKER roles are future scope. V1 must not claim that the Risk Officer workflow is a complete corporate Maker-Checker implementation.

6.3 Role storage

Use *APPUSER*, *ROLE*, and *USERROLE*. *USER_ROLE* provides a controlled many-to-many association so one user may hold multiple roles.

Roles must not be stored as comma-separated text. Public registration cannot assign privileged roles.

A prototype administrator may hold both *SYSTEMADMIN* and *RISKOFFICER*, but every endpoint checks the exact required authority. *SYSTEM_ADMIN* alone cannot approve a payment.

6.4 Customer permissions

A CUSTOMER may:

- Register, authenticate, refresh, and end their own session.
- View their own profile and simulated accounts.
- View their current, reserved, and available balances.
- Create and manage their own beneficiaries.
- Create and authorize transactions from their own eligible accounts.
- View their own transaction history and customer-safe risk explanations.
- Cancel their own eligible transactions before release.
- Complete OTP verification for their own VERY_HIGH transaction.
- Request OTP resend within policy limits.
- View their own notifications and customer-safe transaction timeline.

A CUSTOMER may not:

- Access another customer's account, beneficiary, transaction, or notification.
- Use another customer's source account or beneficiary.
- Select risk tier, score, policy, duration, deadline, or transaction state.
- Approve their own held payment.
- Access the Risk Officer queue or internal notes.
- View raw audit, ledger-reconciliation, password, token, or OTP data.

- Assign roles or unlock an administratively locked account.

6.5 Risk Officer permissions

A RISK_OFFICER may:

- View pending reviews.
- View necessary held-payment information.
- View amount, tier, policy version, explanation, verification result, masked parties, and relevant audit timeline.
- Add internal notes.
- Approve, reject, or request re-verification for an eligible pending review.

A RISK_OFFICER may not:

- Change balances, payment amount, or evaluated risk tier.
- Create customer payments.
- Skip verification.
- Act on a non-pending review.
- Directly force settlement or bypass the transaction state service.
- Edit/delete ledger or audit history.
- View passwords, hashes, OTP values, JWTs, or refresh tokens.
- Edit V1 policy through an unrestricted endpoint.

The officer records a decision. The deterministic transaction service executes the corresponding legal transition.

6.6 System Administrator permissions

A SYSTEM_ADMIN may perform controlled operations such as:

- View user identity and status.
- Lock, unlock, or disable a user.
- Assign or revoke authorized application roles.
- Revoke authentication sessions.
- View permitted system-health and operational failure information.
- Manage controlled prototype provisioning.

A SYSTEM_ADMIN may not:

- View secrets, raw passwords, or OTP values.
- Modify settled payments, balances, ledger history, or audit history.
- Approve a *VERYHIGH* payment unless also assigned RISKOFFICER.
- Grant privileges without an audit event.

Administrative access never bypasses domain invariants.

6.7 Auditor permissions

An AUDITOR has read-only access to approved transaction histories, risk results, review decisions, authentication/security events, ledger entries, reconciliation results, exceptions, and policy versions.

An AUDITOR cannot create, authorize, cancel, verify, approve, reject, settle, assign roles, change policy, modify balances, or edit any audit or ledger record.

6.8 Internal system actor

Schedulers and settlement processors are recorded as SYSTEM actors rather than fake human accounts. Audit entries distinguish human USER actions from automated SYSTEM actions.

Internal services receive only the narrowly required backend capability. They do not authenticate using a hard-coded customer or administrator password.

6.9 Permission matrix

Capability	Customer	Risk Officer	System Admin	Auditor	Internal system
Register as CUSTOMER	Yes	No	No	No	No
View own account	Yes	Review context only	Operational metadata	Audit context	Processing only
Manage own beneficiary	Yes	No	No	Read-only audit	No
Create/authorize payment	Own only	No	No	No	No
Cancel eligible payment	Own only	No	No	No	No
Submit OTP	Own only	No	No	No	No
View risk-review queue	No	Yes	Only if also Risk Officer	Read-only	Processing only
Approve/reject review	No	Yes	Only if also Risk Officer	No	No
Request re-verification	No	Yes	Only if also Risk Officer	No	No
Lock/unlock user	No	No	Yes	No	No
Assign application roles	No	No	Yes	No	No
Revoke sessions	Own logout	Own logout	Yes	Own logout	No
View global audit	No	Relevant context	Permitted operations	Yes	Append only
Modify audit/ledger	No	No	No	No	No
Release expired timer	No	No	No	No	Scheduler only
Perform settlement	No	No	No	No	Settlement service only

6.10 Ownership checks

The backend derives the customer identity from the authenticated principal. It never trusts a submitted customerId.

For customer operations it must verify:

1. The authenticated customer owns the source account.

2. The customer owns the selected beneficiary.
3. The customer owns the selected transaction or notification.
4. The requested action is permitted in the current state.

Ownership-aware queries and service checks prevent horizontal privilege escalation. Role, method, and service checks prevent vertical privilege escalation. A resource owned by another customer should normally appear as 404 rather than confirming that it exists.

6.11 User status and security information

Approved statuses:

- ACTIVE.
- LOCKED.
- DISABLED.

User security information supports failed-login count, lock expiry, last successful login, last failed login, password-change time, security version, and audit timestamps.

A locked or disabled user cannot regain access merely by presenting an older refresh token.

6.12 Authentication sessions

Add AUTH_SESSION to V1 for refresh-token rotation and revocation. Store only a secure hash of the refresh token plus safe session metadata:

- Session ID.
- User ID.
- Refresh-token hash.
- Creation and expiry time.
- Rotation/replacement relationship.
- Revocation time and reason.
- Limited optional session label.

Refresh behaviour:

1. Validate the submitted token.
2. Locate its stored hash.
3. Revoke or rotate the old session.
4. Issue a replacement.
5. Reject reuse of the old token as suspicious.
6. Revoke related sessions according to policy when replay is detected.

Raw refresh tokens are never stored.

6.13 Security version

APP_USER includes a numeric security version. It changes after password reset, disabling, critical role changes, global session revocation, or suspected compromise.

Access tokens carry the version present at issuance. A mismatch invalidates the token.

6.14 Registration and account provisioning

- Public registration always assigns CUSTOMER.
- Registration requests cannot assign a role, balance, or account ID.
- Registration creates a SafePay identity, not a bank balance.
- Demo accounts are provisioned through controlled seed data or an administrator-controlled prototype process.
- There is no public account-creation or opening-balance API.
- A newly registered user without a linked account sees an appropriate no-account state.
- Seeded customers support the complete demonstration journey.

6.15 Sensitive-data and note separation

No role receives raw passwords, password hashes, raw OTPs, OTP hashes, signing secrets, database credentials, or raw refresh tokens.

Account, mobile, email, and beneficiary identifiers are masked where full values are unnecessary.

Customer-safe explanations, customer-visible status messages, internal Risk Officer notes, technical exception details, and audit details are separate representations. Internal notes must never leak through generic entity serialization.

6.16 Defence in depth

Critical authorization is enforced through:

1. Route/security role checks.
2. Service-layer business authorization.
3. Resource ownership checks.
4. Transaction-state validation.
5. Database constraints and conditional updates.
6. Traceable audit events.

6.17 Security audit events

Audit successful/failed login, lock/unlock, logout, refresh rotation, token replay, session revocation, password change, role assignment/removal, disabling, forbidden critical access, and every Risk Officer decision. Secrets and credential values are never logged.

6.18 Oracle and operating-system permissions

Windows administration, Oracle administration, and SafePay application roles are separate.

Approved local prototype workflow:

1. The developer uses their own Windows account to install and run local tools.
2. Connect to FREEPDB1 as Oracle SYSTEM for one-time administrative/bootstrap work.
3. Create and grant a dedicated SAFEPAY_OWNER schema user.
4. Reconnect as SAFEPAY_OWNER to create SafePay objects and run scripts.

The four SafePay roles are application rows and do not restrict the developer's Windows or SQL Developer access.

For a local prototype, *SAFEPAYOWNER may temporarily act as both schema owner and application database login. A production deployment must introduce a restricted SAFEPAYAPP runtime identity.*

7. REST API and Response Contract — Phase 1.1F

7.1 General conventions

- Base path is `/api/v1`.
- Resource URLs use plural nouns.
- JSON fields use camelCase.
- Controllers exchange DTOs, never JPA entities.
- Customer identity comes from authentication.
- Clients cannot submit role, balance, state, risk tier, risk score, policy, duration, or deadline.
- Timestamps use ISO-8601 UTC.
- Financial values preserve decimal precision.
- Responses never expose sensitive internal fields.

7.2 Money and identifiers

Money crosses the API as decimal strings, for example:

```
"amount": "5000.01",  
  "currency": "INR"
```

This prevents JavaScript floating-point alteration. Java converts the value to `BigDecimal`. Oracle sequence-generated identifiers are exposed as JSON strings to avoid JavaScript integer precision loss, even when Java uses `Long` and Oracle uses `NUMBER` internally.

7.3 Time and countdown data

Timestamps use forms such as `2026-09-09T15:30:45.125Z`. A protected transaction may return both `protectedUntil` and `serverTime`. The PWA may display a countdown, but only the backend decides expiry.

7.4 Authentication endpoints

Method	Endpoint	Purpose	Typical success
POST	/api/v1/auth/register	Register CUSTOMER identity	201
POST	/api/v1/auth/login	Authenticate and issue tokens	200
POST	/api/v1/auth/refresh	Rotate refresh session and issue access token	200
POST	/api/v1/auth/logout	Revoke session and clear cookie	204

Registration accepts safe identity and credential fields only. It cannot accept roles, account IDs, or balances. Its response shows CUSTOMER status and whether an account is provisioned. Access tokens are short-lived. The rotating refresh token is sent in an `HttpOnly` cookie and is not exposed to JavaScript.

7.5 Account endpoints

Method	Endpoint	Purpose
GET	/api/v1/accounts	List authenticated customer's accounts
GET	/api/v1/accounts/{accountId}	Retrieve one owned account
GET	/api/v1/accounts/{accountId}/balance	Retrieve current, reserved, and available balance

There is no public account-creation or balance-update endpoint.

7.6 Beneficiary endpoints

Method	Endpoint	Purpose	Typical success
POST	/api/v1/beneficiaries	Create owned beneficiary	201
GET	/api/v1/beneficiaries	List owned beneficiaries	200
GET	/api/v1/beneficiaries/{beneficiaryId}	Retrieve owned beneficiary	200
PATCH	/api/v1/beneficiaries/{beneficiaryId}/status	Enable/disable beneficiary	200

The customer identity never appears in these request bodies.

7.7 Transaction endpoints

Method	Endpoint	Purpose	Idempotency required
POST	/api/v1/transactions	Create payment instruction	Yes
POST	/api/v1/transactions/{transactionId}/authorize	Confirm and process authorization/risk	Yes
GET	/api/v1/transactions	List owned transactions	No
GET	/api/v1/transactions/{transactionId}	Retrieve owned transaction	No
GET	/api/v1/transactions/{transactionId}/risk-explanation	Retrieve safe explanation	No
POST	/api/v1/transactions/{transactionId}/cancel	Cancel eligible owned transaction	Yes

A create request contains source account ID, beneficiary ID, amount string, INR currency, and optional remarks. It cannot contain customer ID, state, tier, score, duration, deadline, approval, or settlement flags.

Authorization validates ownership, state, account, beneficiary, amount, risk policy, and available balance. It reserves funds and produces the appropriate current state. LOW may already be SETTLED when the response returns.

Transaction responses may contain safe source/beneficiary summaries, amount, currency, state, tier, null score, policy version, explanation, protection duration/deadline, server time, canCancel, and lifecycle timestamps.

7.8 Invalid amount submission

The PWA blocks invalid scale or below-minimum values before making an API request. Backend validation still rejects a bypassed request with 400 before creating a transaction, audit event, or transaction exception.

7.9 OTP endpoints

Method	Endpoint	Purpose	Typical success
POST	/api/v1/transactions/{transactionId}/otp	Issue challenge	202
POST	/api/v1/transactions/{transactionId}/otp/verify	Verify challenge	200
POST	/api/v1/transactions/{transactionId}/otp/resend	Replace challenge	202

All require idempotency keys. Raw OTP values never appear in production-style responses or logs. A local demonstration mechanism, if used, must remain isolated from production configuration.

7.10 Risk Officer endpoints

The `/admin` path is retained, but financial actions require `RISK_OFFICER`:

Method	Endpoint	Purpose
GET	<code>/api/v1/admin/risk-reviews</code>	List pending reviews
GET	<code>/api/v1/admin/risk-reviews/{reviewId}</code>	Retrieve review detail
POST	<code>/api/v1/admin/risk-reviews/{reviewId}/approve</code>	Approve eligible review
POST	<code>/api/v1/admin/risk-reviews/{reviewId}/reject</code>	Reject with reason
POST	<code>/api/v1/admin/risk-reviews/{reviewId}/request-verification</code>	Request new verification with reason
POST	<code>/api/v1/admin/risk-reviews/{reviewId}/notes</code>	Add internal note

All state-changing review calls require idempotency keys. Internal notes never appear in customer DTOs.

7.11 Audit and notification endpoints

Method	Endpoint	Purpose
GET	<code>/api/v1/audit-logs</code>	Authorized global audit search
GET	<code>/api/v1/transactions/{transactionId}/audit</code>	Role-appropriate transaction timeline
GET	<code>/api/v1/notifications</code>	List authenticated user's notifications
PATCH	<code>/api/v1/notifications/{notificationId}/read</code>	Mark owned notification read

Customers receive a safe lifecycle timeline. Risk Officers and auditors receive only detail permitted by role.

7.12 Idempotency

Critical mutations send an Idempotency-Key. Its scope includes authenticated actor, endpoint/action, key, and request fingerprint.

- Same key and same request returns the original logical result without re-execution.
- Same key and different request returns 409 Conflict.
- Concurrent duplicates create one effect.
- A replay may include an `Idempotency-Replayed: true` response header.

7.13 HTTP status conventions

Status	Meaning
200	Successful retrieval or completed action with response
201	Resource created
202	OTP issue/resend accepted
204	Successful action with no body
400	Invalid syntax or field validation
401	Missing or invalid authentication
403	Authenticated role lacks permission
404	Resource absent or not visible to actor
409	Illegal state, idempotency conflict, or concurrency loss
422	Structurally valid request violating a business rule
429	Authentication or OTP rate limit exceeded
500	Unexpected safely handled server error
503	Required service temporarily unavailable

Examples of 422 include insufficient available balance, disabled beneficiary, inactive source account, and incorrect/expired OTP.

7.14 Standard error model

Errors use RFC 7807 ProblemDetail with appropriate type, title, status, safe detail, instance, stable errorCode, traceId, timestamp, and optional fieldErrors.

Errors never expose SQL, credentials, stack traces, internal entity details, tokens, OTP values, or internal review notes.

7.15 Pagination

Paged responses contain items, page, size, totalElements, totalPages, first, and last.

- First page is 0.
- Default size is 20.
- Maximum size is 100.
- Invalid or unsupported sort fields are rejected.

This applies to transaction, review, notification, and audit collections as appropriate.

7.16 Authentication transport and browser security

- Protected APIs use `Authorization: Bearer <access-token>`.

- Refresh token uses HttpOnly, appropriately scoped, expiring cookie storage.
- Secure is required over HTTPS; local profile may accommodate local HTTP only.
- SameSite and Origin protections apply to cookie-using operations.
- Production uses same-origin frontend/backend deployment.
- Local CORS permits only explicit development origins.
- Wildcard credentialed CORS is prohibited.

7.17 Correlation IDs

Every request has an X-Correlation-ID generated or safely accepted by the backend. It links API request, application logs, audit events, exceptions, and settlement attempts. It is not an authentication credential.

7.18 WebSocket contract

The protected `/ws` channel may announce transaction and review status changes. It cannot directly settle or mutate a payment. After disconnect or reconnect, REST retrieves authoritative state.

7.19 OpenAPI

The approved REST contract will produce OpenAPI documentation. Swagger UI may be available for controlled local learning/testing and restricted or disabled in production.

7.20 API security invariants

- Never trust a submitted customer ID.
 - Never accept a client-selected role, state, tier, score, or deadline.
 - Never return JPA entities directly.
 - Never expose another user's data.
 - Never use binary floating point for money.
 - Never mutate financial state without state and ownership checks.
 - Never rely on frontend validation alone.
 - Never duplicate a financial effect because of retry.
 - Never expose sensitive fields through generic serialization.
-

8. Final Conflict Register and Sign-off — Phase 1.1G

8.1 Final V1 feature classification

Area	V1 decision
Customer authentication	Included
JWT access authentication	Included
Rotating refresh sessions	Included
Simulated customer accounts	Included and pre-provisioned
Beneficiary management	Included
Payment creation and confirmation	Included
Amount-only risk classification	Included
LOW immediate settlement	Included
MEDIUM/HIGH Undo protection	Included
VERY_HIGH OTP and Risk Officer hold	Included
Fund reservation	Included
Double-entry ledger	Included
Idempotency and concurrency control	Included
Audit and transaction exceptions	Included
In-app notifications and WebSocket updates	Included
Oracle JET responsive PWA	Included
Device/location and weighted scoring	Excluded
AI risk assistance	Future scope
Complete Maker-Checker	Future scope
Post-settlement dispute workflow	Future scope
Real payment-rail integration	Excluded

8.2 Consolidated conflict register

Baseline item	Approved V1 resolution
Weighted multi-signal score	Deterministic amount-only classification
Mandatory numeric risk score	Nullable riskScore
Sub-rupee payment minimum	Minimum ₹1.00
Earlier HIGH upper boundary	Latest approved ₹1,00,000.00 boundary
Device/location data and rules	Removed completely from V1
Beneficiary-age/frequency/anomaly signals	Deferred to V2
AI recommendation	Future read-only/advisory scope
Corporate Maker-Checker	Deferred as a complete feature
VERY_HIGH path using timed protection	Untimed verification and Risk Officer hold
APPROVED/REJECTED payment states	Separate risk-review statuses
Editable V1 policy UI	Deferred; policy is versioned and seeded
User safety settings	Deferred
Post-settlement disputes	Deferred
Real SMS/email	Excluded; isolated local simulation only
Live rail settlement	Excluded
Unversioned API	<code>/api/v1</code>
Direct entity serialization	DTO-only boundary
Java alternatives	Java 21 LTS
Domain-oriented package proposal	Approved layered <code>com.ofss</code> structure
React/Vue alternatives	Oracle JET MVVM with Knockout and TypeScript
Browser-authoritative countdown	Server/database-authoritative deadline
Balance check only at settlement	Approved reservation model
Direct external-beneficiary ledger credit	Internal outbound-clearing credit
Mutable ledger history	Append-only entries and compensating corrections
Oracle SYSTEM owning application objects	Dedicated SAFEPAY_OWNER schema
Application roles treated as Oracle roles	Separate SafePay business-role records

8.3 OTP policy defaults

Policy	Approved V1 value
Format	6 numeric digits
Validity	5 minutes
Failed verification attempts	Maximum 3
Resend cooldown	30 seconds
OTP issues in one verification cycle	Maximum 3 total: initial plus two resends
Storage	Secure hash only
Reuse	Prohibited
Previous OTP after resend	Immediately invalid
Exhausted policy result	Transaction becomes CANCELLED
Customer cancellation	Allowed while awaiting verification

OTP policy values are configuration/policy data, not scattered constants.

8.4 Transaction authorization simplification

- Customer must have an authenticated session.
- Payment review displays the full safe instruction.
- Customer explicitly selects Confirm and Authorize.
- Backend revalidates authentication, ownership, account, beneficiary, amount, and state.
- V1 does not create a separate stored transaction PIN lifecycle.
- VERY_HIGH retains its dedicated OTP step.

This remains explicit simulated authorization rather than automatic authorization during creation.

8.5 Notification boundary

- In-app notifications are the authoritative V1 notification channel.
- WebSocket events provide immediate updates.
- REST retrieves stored notifications and authoritative status.
- No real email or SMS is sent.
- Local OTP demonstration is isolated from production-style configuration.
- Push notifications are outside the critical V1 path unless separately approved.

8.6 PWA invariants

The frontend uses Oracle JET MVVM, Knockout, and TypeScript. It must remain usable on approved mobile and desktop sizes.

The PWA never calculates authoritative tier/deadline, stores access tokens persistently, caches sensitive API data, or queues financial mutations offline. REST restores state after reconnection. Simulated language is mandatory around balance and settlement.

8.7 V2 extension boundary

V2 may introduce weighted factors, beneficiary signals, velocity, amount anomaly, device, location, configurable thresholds, user safety settings, full Maker-Checker, disputes, analytics, and read-only AI assistance.

The transaction orchestration consumes a stable risk-result contract. V1 AmountRiskEngine can later be replaced or complemented without rewriting state, protection, settlement, audit, and security logic.

Device/location work requires a separately approved consent, retention, security, schema, API, frontend, and testing cycle. V1 contains no dormant columns for it.

9. Implementation Reference Rules

Before implementing any SafePay unit:

1. Re-read the latest user instruction.
2. Read root AGENTS.md.
3. Read this Decision Register.
4. Consult the updated implementation plan.
5. Consult the PRD and other resources for supporting detail.
6. Check the conflict register before restoring any baseline feature.
7. Explain the next manual action and receive approval at the required checkpoint.

The implementation order remains:

1. Prerequisites, project setup, and Oracle database.
2. Backend and REST APIs in entity/DAO/service/controller increments.
3. Complete backend compilation, logic, concurrency, security, and API testing.
4. Oracle JET PWA frontend.
5. Deployment and server settings.

No phase is considered complete while it contains an unexplained error. A dependency may be carried forward only when its future phase is named explicitly.

10. Approval Register

Phase	Contract	Status
1.1A	Authority, scope, and change control	APPROVED
1.1B	Amount-risk and protection windows, including later numeric amendments	APPROVED
1.1C	Transaction state-transition contract	APPROVED
1.1D	Reservation, settlement, and immutable double-entry ledger	APPROVED
1.1E	Users, roles, permissions, sessions, and ownership	APPROVED
1.1F	REST API and response conventions	APPROVED
1.1G	Conflict register, OTP defaults, authorization, and notification boundary	APPROVED

Phase 1.1 overall status: APPROVED AND FROZEN FOR IMPLEMENTATION.

“Frozen” means this is the currently approved implementation contract. The user may change it later through the precedence and change-control procedure in this document.

11. Future Approved-Change Log

For every later change, append:

Date	Requested change	Superseded decision	Impacted areas	Approval status
—	—	—	—	—